

Unsupervised learning

Friday march 14

Today

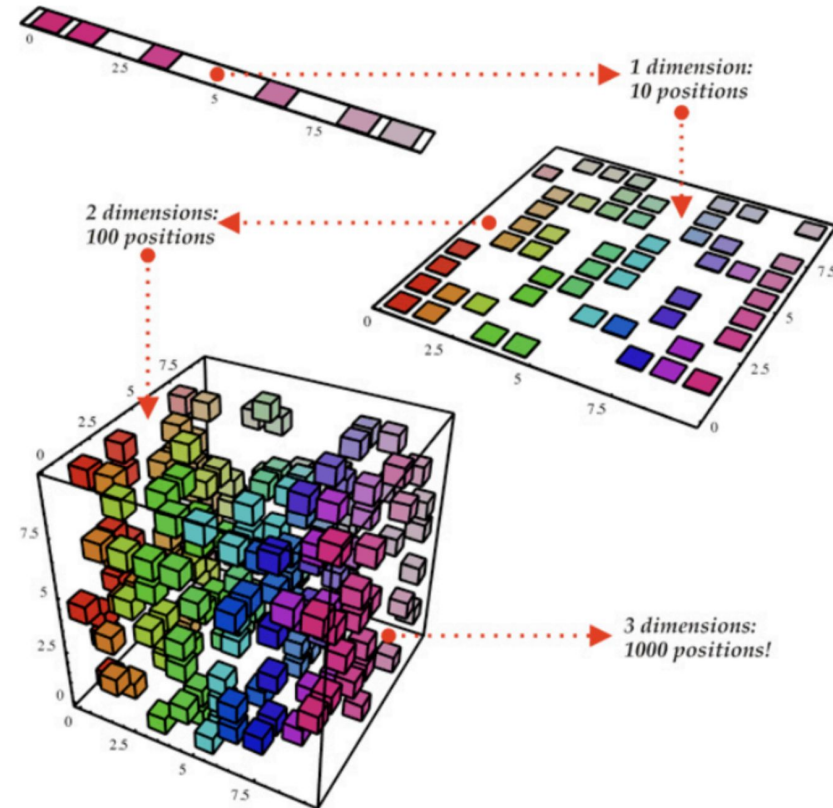
Unsupervised learning - No ground truth dataset

- Dimension reduction
- Clustering

Dimension Reduction: PCA and Beyond

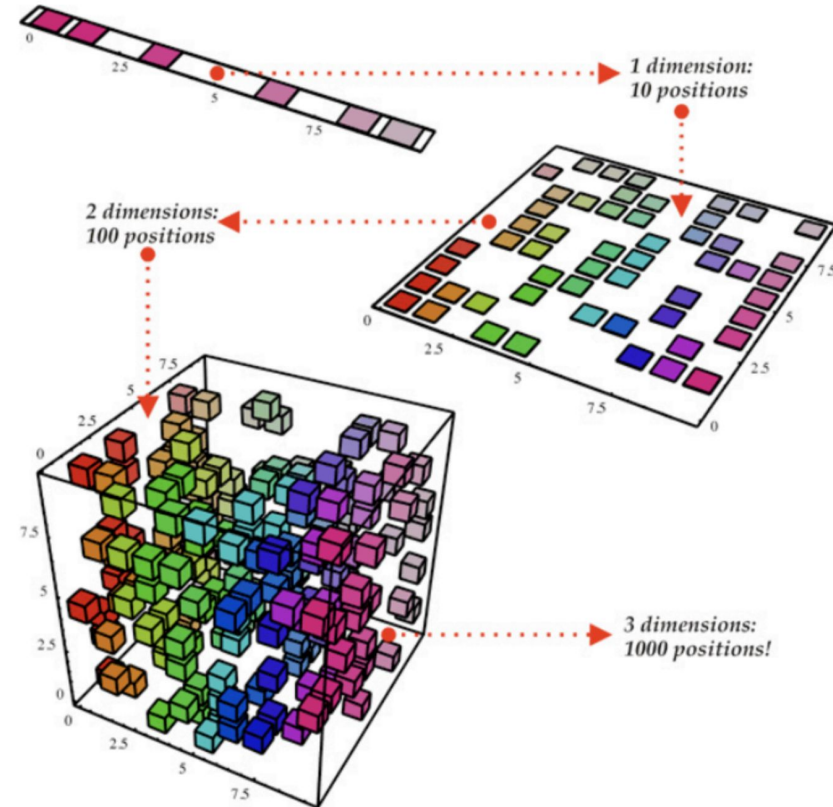
The Problem of High Dimensionality

- Data often contains many features (dimensions)
- The "curse of dimensionality": performance degrades as dimensions increase
- Challenges with high-dimensional data:
 - Increased computational complexity
 - Difficulty in visualization
 - Sparsity of data points in high-dimensional space
 - Risk of overfitting



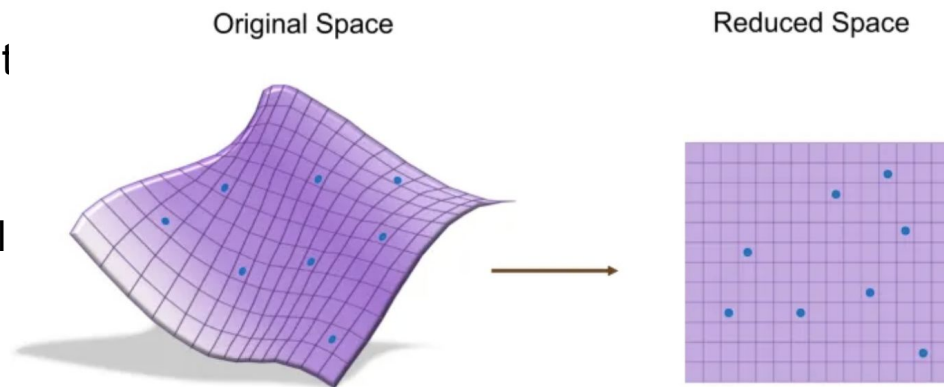
The curse of High Dimensionality - Cars dataset

- The cars93 dataset has several categorical features with many values: Model, Make, ...
- if you one hot encode these features you end up with new features
- the total number of features ends up higher than the number of samples



Why Dimension Reduction?

- Simplifies data while preserving important information
- Removes redundant/correlated features
- Enables **visualization** of high-dimensional data



Linear Dimension Reduction: PCA

Principal Component Analysis (PCA) is a fundamental technique

- Projects data onto a lower-dimensional linear subspace
- The new dimensions (principal components) are:
 - Orthogonal to each other
 - Ordered by amount of variance explained
 - Linear combinations of original features

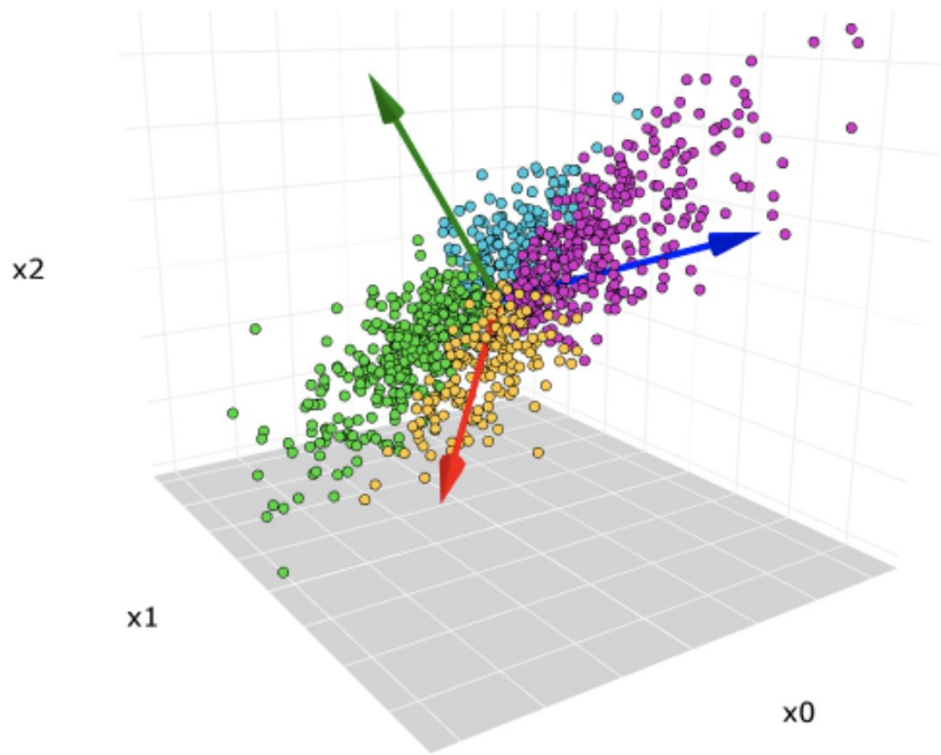
How PCA Works

- Steps:
 1. Center the data (subtract mean)
 2. Compute covariance matrix
 3. Find eigenvectors and eigenvalues of covariance matrix
 4. Select top k eigenvectors (by eigenvalue magnitude)
 5. Project data onto these eigenvectors

Linear Dimension Reduction: PCA

PCA transforms the original features into a new set of uncorrelated variables called principal components.

- Identifies directions (principal components) with maximum variance
- Projects data onto these components
- Naturally eliminates redundancy by creating orthogonal components
- Allows you to keep only the components that explain most of the variance



- **Singular Value Decomposition (SVD):** Any matrix $X \in \mathbb{R}^{n \times d}$ can be decomposed as: $X = U\Sigma V^T$ where:
 - $U \in \mathbb{R}^{n \times n}$ contains left singular vectors (orthonormal)
 - $\Sigma \in \mathbb{R}^{n \times d}$ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$
 - $V \in \mathbb{R}^{d \times d}$ contains right singular vectors (orthonormal)
- **Connection to PCA:**
 - For centered data matrix X , the covariance matrix is $C = \frac{1}{n-1} X^T X$
 - Using SVD: $C = \frac{1}{n-1} V \Sigma^T \Sigma V^T$
 - The eigenvectors of C are the right singular vectors V
 - The eigenvalues of C are $\lambda_i = \frac{\sigma_i^2}{n-1}$
 - Principal components are given by $XV = U\Sigma$
 - The proportion of variance explained by the i -th component: $\frac{\lambda_i}{\sum_j \lambda_j} = \frac{\sigma_i^2}{\sum_j \sigma_j^2}$
- **Dimension Reduction:** Keep only the top k components by using V_k (first k columns of V) $X_k = XV_k = U\Sigma V^T V_k = U_k \Sigma_k$

PCA in scikit -learn

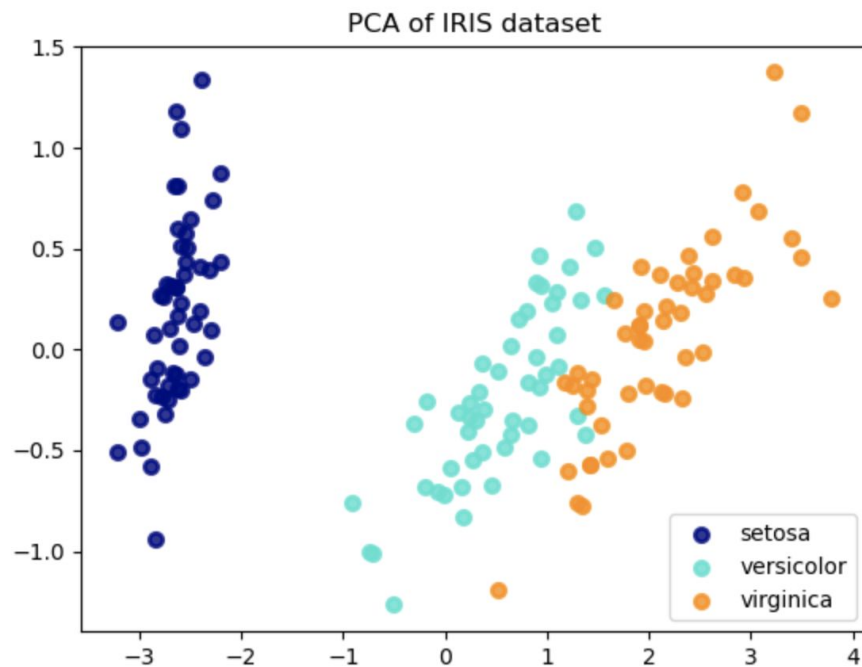
<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>

```
from sklearn.decomposition import PCA
```

```
X = load_iris().data
```

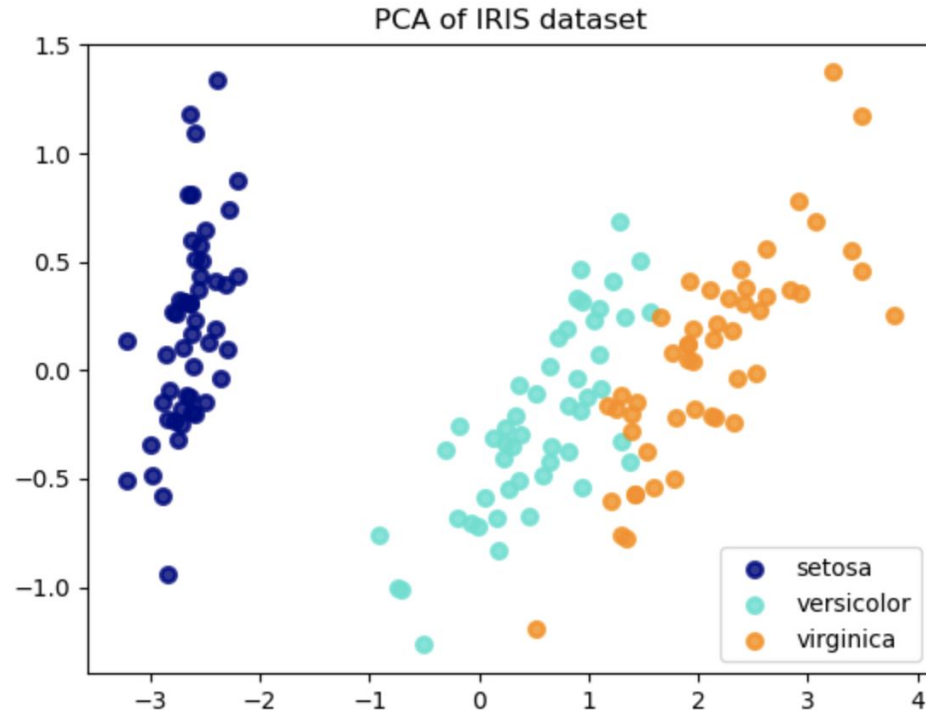
```
pca = PCA(n_components=2)  
pca.fit(X)
```

```
print(pca.explained_variance_ratio_)  
print(pca.singular_values_)
```



PCA in scikit -learn

https://colab.research.google.com/drive/1I0EZ2GG7U5pzk6QZJNrBW8_NbhG_O6Jh#scrollTo=ALL-K7eSzMxe



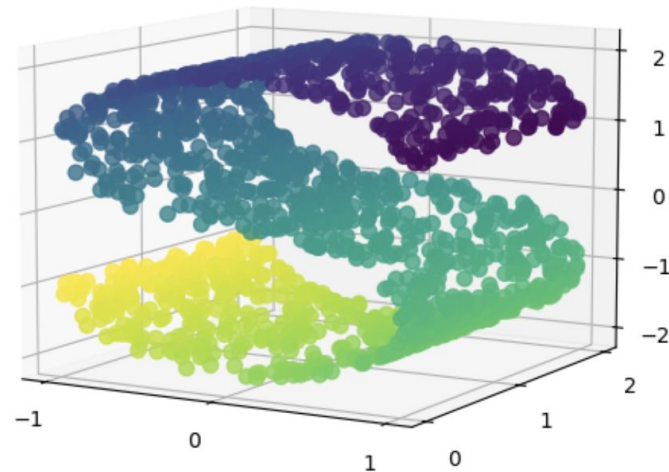
Linear or Non Linear

Limitations of PCA

- Only captures linear relationships
 - Cannot properly represent data on curved manifolds
 - May miss important non-linear patterns
 - All features contribute to all principal components
 - Difficult to interpret components in some domains

Linear vs. Non-linear Data

- **Linear structure:** Data points lie near a linear subspace
 - Example: Points scattered around a line or plane
 - PCA works well for these datasets
- **Non-linear structure:** Data points lie on or near curved manifolds
 - Example: Swiss roll, S-curve, or concentric circles
 - PCA fails to capture the intrinsic structure



Beyond Linear: Manifold Learning

- Manifold learning techniques capture non-linear structure
- Key idea: preserve local relationships while reducing dimensions
- Popular methods:
 - t-SNE (t-Distributed Stochastic Neighbor Embedding)
 - UMAP (Uniform Manifold Approximation and Projection)
 - Isomap (Isometric Mapping)
 - LLE (Locally Linear Embedding)
 - Autoencoders (neural network approach)

Dimension Reduction: T-SNE



Core Process of t-SNE

Similarity Calculation in High Dimensions

- For each pair of points, t-SNE calculates their similarity using Gaussian distributions
- The similarity represents how likely points are to be neighbors
- Points close together have high similarity; distant points have low similarity

Mapping to Low Dimensions

- t-SNE creates a corresponding low-dimensional map (typically 2D or 3D)
- In the low-dimensional space, similarities are calculated using t-distributions (heavier tails than Gaussian)
- The t-distribution helps address the "crowding problem" by allowing moderate distances in low dimensions

Optimization via Gradient Descent

- t-SNE minimizes the difference between the similarity distributions in high and low dimensions
- This difference is measured using Kullback-Leibler (KL) divergence
- The algorithm iteratively adjusts the low-dimensional positions to minimize this KL divergence

Kullback-Leibler divergence is a measure of how one probability distribution differs from another reference distribution:

- **Mathematical Definition:**
$$KL(P || Q) = \sum P(x) \log(P(x)/Q(x))$$
- **Interpretation:** KL divergence measures the information lost when distribution Q is used to approximate distribution P

Core Process of t-SNE

In t-SNE, similarities between points in high-dimensional space are calculated using adaptive Gaussian distributions:

- **Distance Calculation:** Compute Euclidean distances between all pairs of points.
- **Gaussian Probability:** For each point, **convert distances to probabilities** using a Gaussian distribution centered on that point.
- **Adaptive Bandwidth:** The spread (σ) of each Gaussian is automatically determined based on the **user-defined "perplexity"** parameter, which roughly specifies how many neighbors each point should consider.
- **Symmetrization:** Average the conditional probabilities to create symmetric similarity values that serve as the target distribution for the low-dimensional embedding.

This approach effectively captures local neighborhood relationships while adapting to varying data densities, allowing t-SNE to reveal clusters in complex datasets.

In short

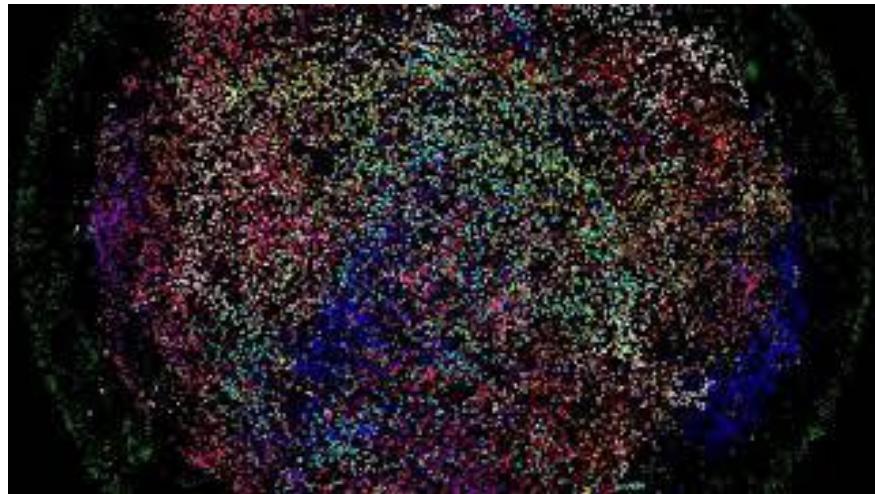
- It uses Gaussian distributions to convert distances between points into probabilities
- Each point gets its own Gaussian with width adjusted by the perplexity parameter
- Points that are close in high dimensions receive high probability of being neighbors
- These probabilities form a target distribution that the low-dimensional embedding attempts to match
- The adaptive nature of the Gaussians helps t-SNE handle varying data densities and find meaningful clusters

T-SNE

- Non-linear dimension reduction technique focused on preserving local relationships
- Converts high-dimensional similarities to joint probabilities and minimizes KL divergence
- Uses t-distribution in low-dimensional space to avoid the "crowding problem"

Key Properties

- Excels at revealing clusters and local structure in data
- Preserves neighborhood relationships rather than global structure
- Results depend on perplexity parameter (typical range: 5-50)
- Non-deterministic: different runs produce different embeddings
- Computationally intensive compared to linear methods like PCA

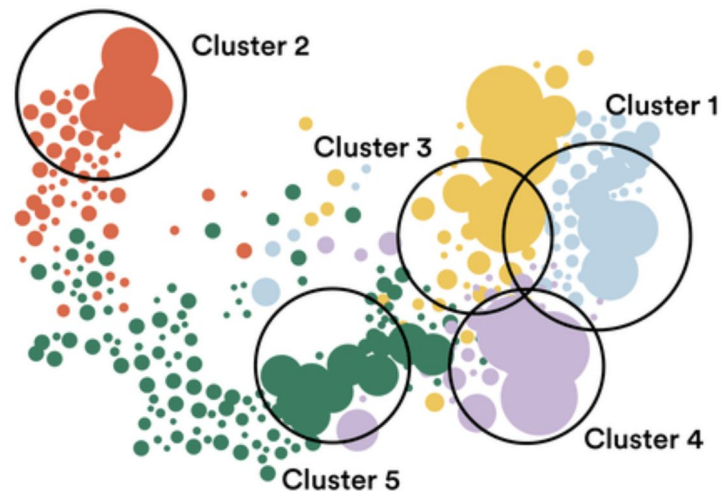


Clustering

k-means

Clustering use cases

- **Customer segmentation:** Group customers with similar purchasing behaviors to create targeted marketing campaigns and personalized experiences.
- **Anomaly detection:** Identify unusual patterns that don't conform to expected behavior, critical for security monitoring and fraud prevention.
- **Document organization:** Automatically group similar documents for better information retrieval and knowledge management.
- **Image segmentation:** Partition images into regions that correspond to different objects or parts, essential for computer vision.
- **Genomic analysis:** Identify groups of genes with similar expression patterns to discover biological functions.
- **Recommendation systems:** Group users with similar preferences to suggest relevant products, content, or connections.

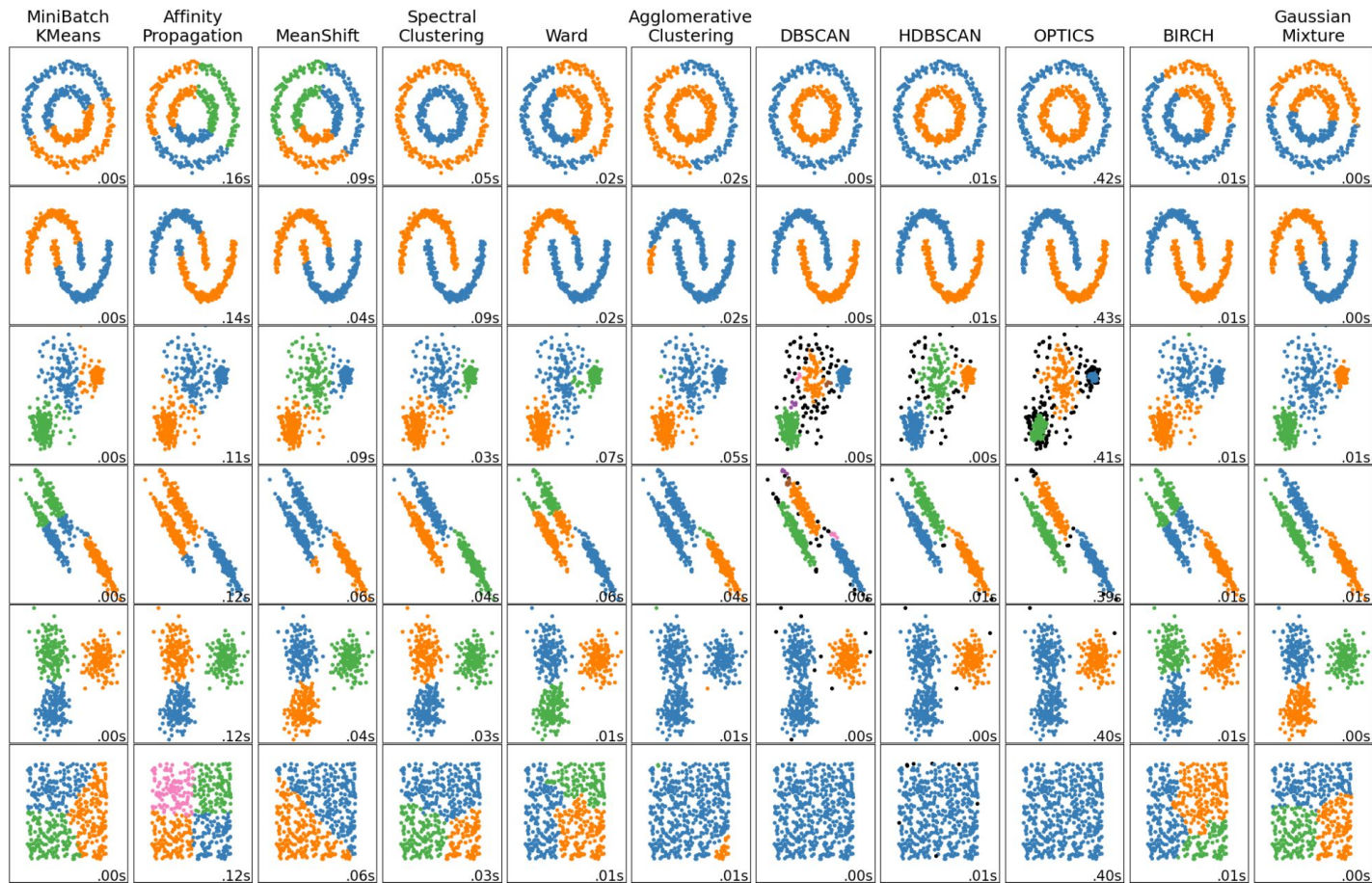


Principles of Clustering

- **Unsupervised learning:** Clustering doesn't require labeled data; it discovers inherent structures.
- **Similarity-based grouping:** Objects in the same cluster should be more similar to each other than to objects in other clusters.
- **Distance metrics:** Algorithms typically measure similarity using distance functions (Euclidean, Manhattan, cosine, etc.) appropriate to the data type.

- **Internal evaluation:** Clusters are assessed based on **compactness** (how close points within clusters are) and **separation** (how distinct different clusters are).
- Silhouette Score, elbow method

Clustering algos in sklearn

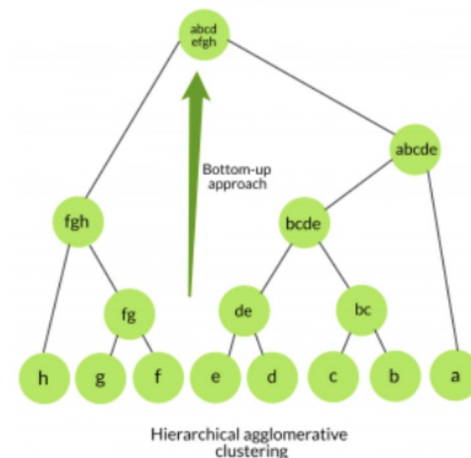


Main Algorithms

Hierarchical Methods

- **Agglomerative clustering:**
 - Starts with each point as its own cluster and merges the closest pairs
 - Creates a dendrogram showing the entire clustering hierarchy
 - Doesn't require specifying number of clusters in advance
 - Linkage criteria (single, complete, average, Ward) determine merge strategy
- **Divisive clustering:**
 - Begins with all points in one cluster and recursively splits
 - Less common but useful for certain applications
 - Can reveal hierarchical structure in data

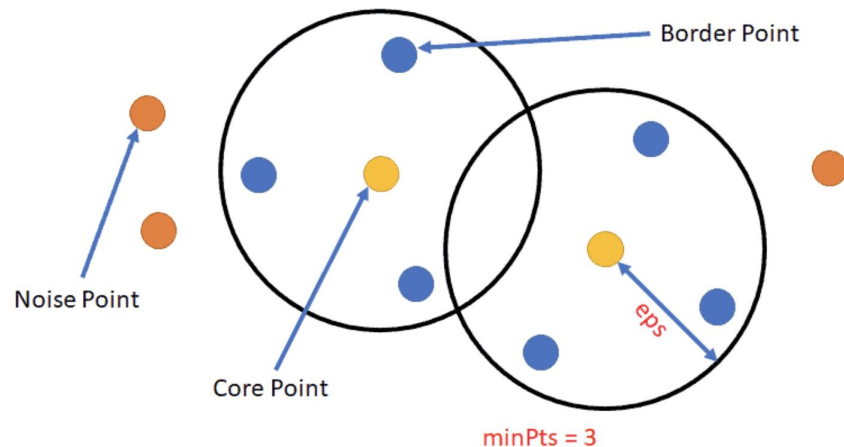
Agglomerative Clustering



Main Algorithms

Density-Based Methods

- **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise):
 - Defines clusters as dense regions separated by sparse regions
 - Automatically discovers clusters of arbitrary shapes
 - Robust to outliers, which are labeled as noise
 - Doesn't require pre-specifying number of clusters



Distribution-Based Methods

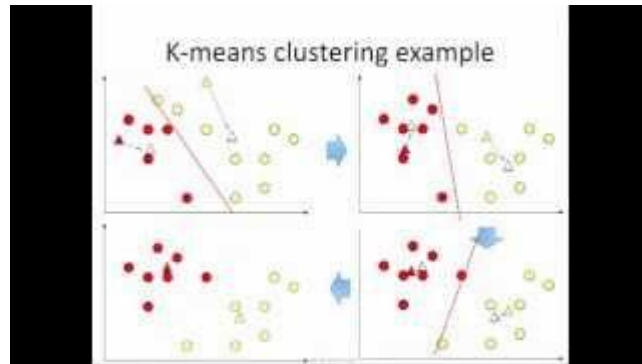
- **Gaussian Mixture Models (GMM):**
 - Assumes data points are generated from a mixture of several Gaussian distributions
 - Soft assignment: points have probabilities of belonging to each cluster
 - Can capture elliptical clusters and overlapping distributions
 - Uses Expectation-Maximization (EM) algorithm for optimization

k-means

K-means

Partitioning Methods

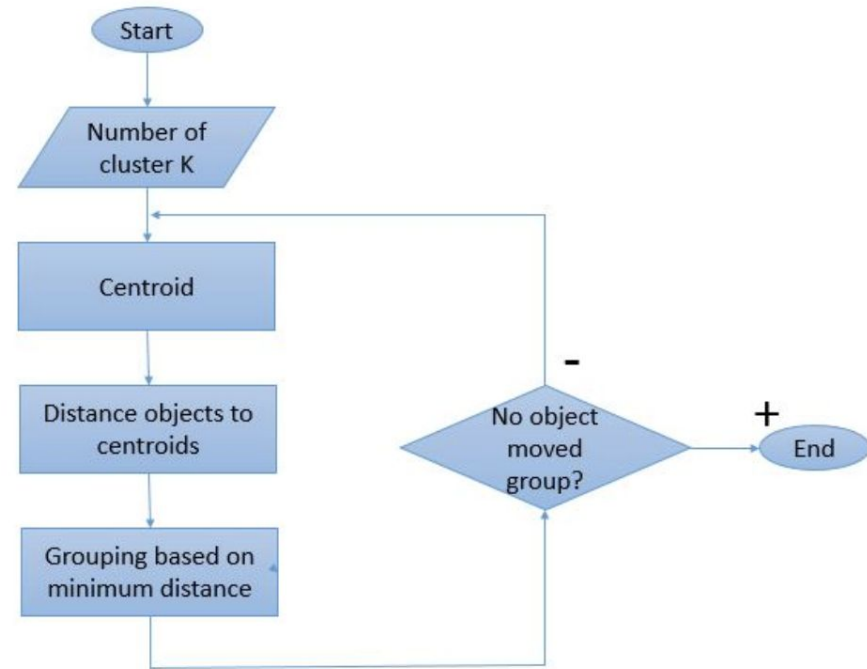
- **K-means:**
 - Divides data into k non-overlapping subsets by minimizing within-cluster variance
 - Represents each cluster by its centroid (mean of points)
 - Fast and scalable, but assumes spherical clusters



K-means

Algorithm Steps

1. **Initialization:** Randomly select k data points as initial centroids (or use k-means++ for smarter initialization)
2. **Assignment step:** Assign each data point to the nearest centroid, forming k clusters
3. **Update step:** Recalculate centroids as the mean of all points assigned to that cluster
4. **Iteration:** Repeat steps 2-3 until centroids stabilize or maximum iterations reached



<https://www.naftaliharris.com/blog/visualizing-k-means-clustering/>

K-means

- **Computational efficiency:** $O(tknd)$ where t =iterations, k =clusters, n =points, d =dimensions
- **Guaranteed convergence:** Will always converge to a local optimum
- **Scalability:** Can be implemented in parallel and with mini-batches for large datasets
- **Interpretability:** Centroids provide meaningful representations of clusters

Limitations

- **Shape constraints:** Only identifies spherical clusters; struggles with complex shapes
- **Equal size assumption:** Tends to create clusters of similar sizes and variances
- **Outlier sensitivity:** Outliers can significantly distort cluster centroids
- **Feature scaling:** Results highly dependent on feature scaling; standardization often required
- **Empty clusters:** Can result in empty clusters if poorly initialized

K-means

Practical Considerations

- **Choosing k:**
 - Elbow method: Plot WCSS vs. k and look for an "elbow" point
 - $WCSS = \sum \text{distance}^2(\text{point}, \text{centroid})$ for all points
 - Measures how closely points are clustered around their centroids
 - Plotting K vs. WCSS produces a line that resembles an arm
 - Silhouette scores: Measure how similar objects are to their own cluster vs. other clusters
 - Gap statistic: Compare intra-cluster variation to expected under null reference
- **Initialization sensitivity:**
 - K-means++ improves initial centroid selection by choosing points far from each other
 - Multiple runs with different initializations can find better solutions

The Elbow Method for Selecting K

How It Works

- **Purpose:** Determine the optimal number of clusters (k) for K-means
- **Process:** Plot the sum of squared distances (inertia) against increasing k values
- **Principle:** As k increases, inertia decreases - points are closer to their centroids
- **Decision point:** Look for the "elbow" where adding more clusters yields diminishing returns

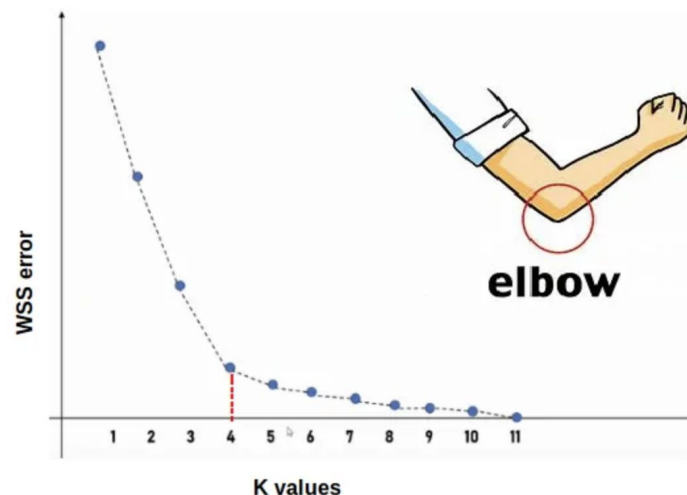
Visualization

- X-axis: Number of clusters (k)
- Y-axis: Inertia (sum of squared distances from points to centroids)
- The curve typically resembles an arm, with the "elbow" indicating optimal k

Limitations

- Interpretation can be subjective when there's no clear elbow
- May suggest suboptimal k when clusters have varying densities or sizes
- Should be used alongside other methods (silhouette score, gap statistic)

Elbow method



K-means - silhouette score

Silhouette Score

- Measures how well each object fits within its assigned cluster
- Range: -1 to +1
- Formula: $(b-a)/\max(a,b)$ where:
 - a = average distance to points in same cluster
 - b = average distance to points in nearest neighboring cluster

Interpretation of Scores

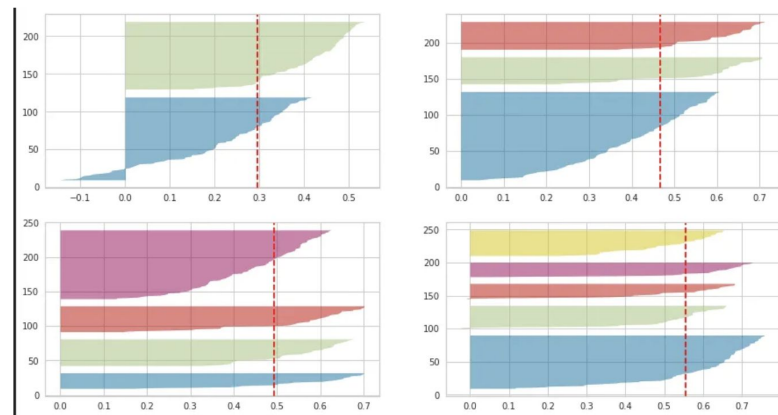
- **Near +1:** Object well-clustered
- **Near 0:** Object on border between clusters
- **Near -1:** Object likely assigned to wrong cluster

Silhouette Graph

- x-axis: silhouette coefficient values
- y-axis: clustered objects grouped by cluster

How to Interpret the Graph

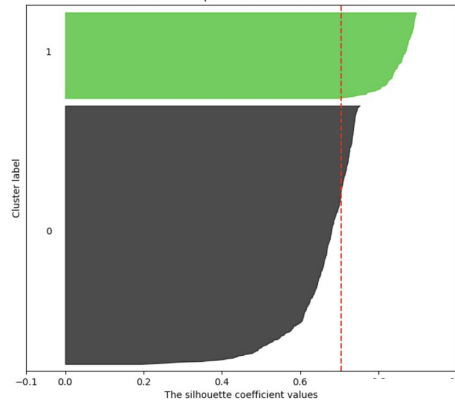
- **Cluster Width:** Wider clusters indicate better separation
- **Cluster Position:** Clusters further right have better cohesion
- **Cluster Size:** Height shows number of objects per cluster
- **Cluster Shape:** Uniform width suggests consistent clustering
- **Average Line:** Vertical line showing average silhouette score



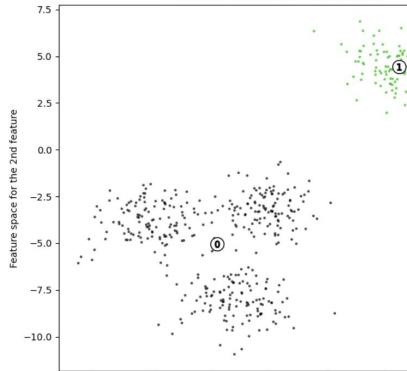
Optimal number of cluster K ?

In many clustering methods you need to set the number of clusters beforehand. It is difficult to clearly determine the optimal number of clusters

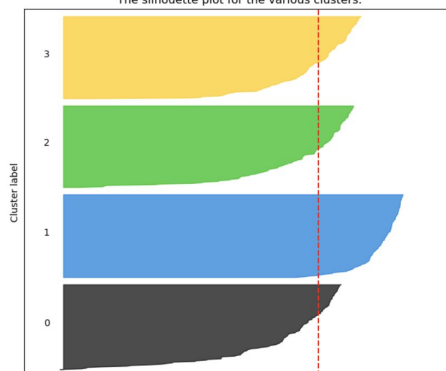
The silhouette plot for the various clusters.



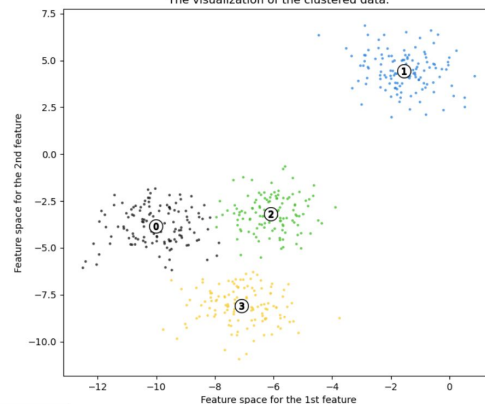
The visualization of the clustered data.



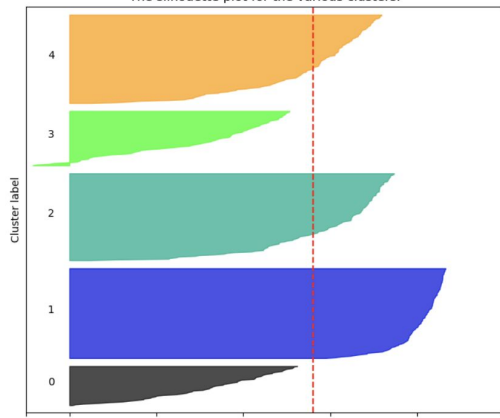
The silhouette plot for the various clusters.



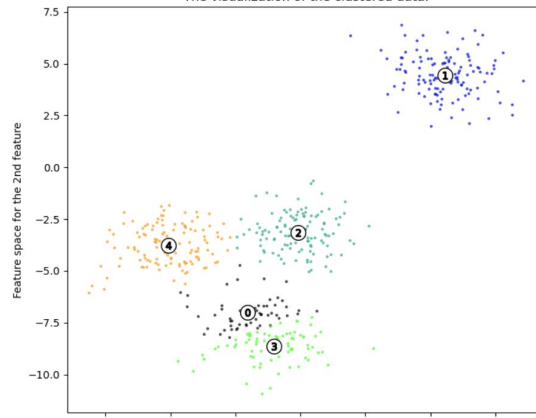
The visualization of the clustered data.



The silhouette plot for the various clusters.



The visualization of the clustered data.



K-means - silhouette method

The Silhouette Coefficient is calculated using

- the mean intra-cluster distance (a)
- the mean nearest-cluster distance (b)

for each sample.

<https://scikit-learn.org/stable/modules/clustering.html#silhouette-coefficient>

https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html

Practice

<https://colab.research.google.com/drive/1JM7QQsmcnY4PeDOH2Xg0nqR9UI2RyfUf>